F.CSM302 ҮЙЛДЛИЙН СИСТЕМ

БИЕ ДААЛТ 1 — ТАЙЛАН

The Sleeping Teaching Assistant

Thread, Semaphore, Mutex ашигласан synchronization симуляци ба Tkinter дүрслэл

Хуваарилалтын код: **II-4-10**

№	Нэр	Оюутны код
1	А.Төгөлдөр	B242270001
2	Ц.Цогтжаргал	B241940005
3	Д.Отгон-Эрдэнэ	B222270083
4	О.Цэрэнлхавга	B232270113
5	Г.Түвшинсайхан	B242270085

GitHub: https://github.com/B242270001/II-4-10_SleepingTeachingAssistant

Улаанбаатар хот, 2026 он

Хураангуй

Энэхүү бие даалтаар Silberschatz-ийн *Operating System Concepts* (10th edition) сурах бичгийн 7-р бүлгийн “The Sleeping Teaching Assistant” програмчлалын төслийг Python хэл дээр хэрэгжүүлэв. Бодлогод нэг багшийн туслах (TA) болон хязгаарлагдмал тооны хүлээлгийн сандал, олон оюутан оролцоно. Оюутан байхгүй үед TA унтаж, оюутан ирэхэд сэрнэ. Сандал дүүрэн үед ирсэн оюутан буцна.

Төслийг хоёр хэсэгт хуваасан. **Core** (`ta_core.py`) нь TA болон оюутан бүрийг бодит thread болгон ажиллуулж, хүлээлгийн дарааллыг Lock (mutex)-ээр хамгаалж, унтах ба сэрэхийг Semaphore-оор хэрэгжүүлсэн. Core нь UI-аас хамааралгүй бөгөөд thread бүр өөрийн үйлдлийг event болгон дарааалалд (`queue.Queue`) хийдэг. **UI** (`ui.py`) нь Tkinter дээр бүтсэн dashboard бөгөөд тэр event-үүдийг уншиж TA, сандал, оюутныг хөдөлгөөнт дүрслэлээр, мөн тоолуур, график, event түүхээр бодит цагт харуулна. Хэрэглэгч сандал, оюутны тоо, ирэх ба үйлчилгээний хугацааг тохируулж, START, PAUSE, STEP, RESET удирдлагаар симуляцийг хянана.

Туршилтыг гурван түвшинд хийсэн: багийн 11 автомат тест, 27 нэмэлт edge case тест, 11 UI сценари. Бүх тест тэнцсэн. Synchronization-ы гол шинж болох FIFO дарааалал, сандлын тоо хэтрэхгүй байх, TA нэг дор нэг оюутанд үйлчлэх, deadlock-гүй байх нь бүх тохиргоонд, түүний дотор дээд ачаалалд (20 сандал, 50 оюутан) биелсэн. Илэрсэн гол сул тал нь ачаалал ихтэй үед UI-ийн дүрслэл core-оос хоцрох явдал юм.

Түлхүүр үг: synchronization, thread, semaphore, mutex, Sleeping Teaching Assistant, Python, Tkinter.

1. Удиртгал

1.1 Сэдвийн холбогдол

Орчин үеийн үйлдлийн систем олон процесс, thread-ийг зэрэг ажиллуулдаг. Эдгээр нь нийтийн нөөц (санах ой, дараалал, төхөөрөмж)-ийг хуваалцах үед синхрончлол алдагдвал race condition, deadlock зэрэг ноцтой алдаа гарна. Тиймээс mutex ба semaphore-оор нөөцийг зөв хамгаалж, thread-үүдийг зөв унтуулж сэрээх нь үйлдлийн системийн гол ойлголтуудын нэг юм. Sleeping Teaching Assistant бодлого нь эдгээр ойлголтыг өдөр тутмын нэг ойлгомжтой нөхцөлөөр харуулдаг сонгодог жишээ учраас сургалтад өргөн хэрэглэгддэг.

1.2 Асуудлын тодорхойлолт

Нөхцөл дараах байдалтай:

- TA-ийн өрөөнд нэг сандал (TA), коридорт N хүлээлгийн сандал байна.
- Оюутан байхгүй бол TA унтана.
- Оюутан ирэхэд TA унтаж байвал сэрээнэ. TA завгүй бол сул сандалд суугаад хүлээнэ. Сул сандал байхгүй бол буцаж явна, хэсэг хугацааны дараа дахин ирнэ.
- Оюутнууд FIFO дарааллаар үйлчлүүлнэ.

Хэрэгжүүлэлтэд хоёр төрлийн алдаанаас сэргийлэх шаардлагатай. Нэгдүгээрт, хүлээлгийн дараалал болон TA-ийн төлөвийг олон thread зэрэг өөрчлөхөөс (race condition). Хоёрдугаарт, TA унтсан атлаа оюутан хүлээж үлдэх, эсвэл оюутан TA-г сэрээж чадалгүй гацах (lost wakeup, deadlock) байдлаас.

1.3 Зорилго, зорилт

Зорилго: Sleeping Teaching Assistant бодлогыг thread, semaphore, mutex ашиглан зөв хэрэгжүүлж, ажиллагааг нь харагдахуйц дүрслэх.

Зорилтууд:

- UI-аас хамааралгүй, туршиж болохуйц core симуляци бичих (`ta_core.py`).
- Core-ийн бодит thread-үүдийн төлөвийг харуулсан интерактив Tkinter dashboard хийх (`ui.py`).
- Оролтыг шалгаж, буруу утгыг ойлгомжтой мессежээр мэдэгдэх.
- START, PAUSE, STEP, RESET удирдлага, event түүхийг шүүх, хайх, CSV болгон хадгалах боломж олгох.
- Автомат тест, edge case тест, UI сценариар зөв байдлыг баталгаажуулах.

1.4 Хэрэгжүүлсэн шийдлийн товч

Шийдэл нь номын сонгодог загвартай ижил. Core-ийн synchronization объектууд:

Объект	Төрөл	Үүрэг
`mutex`	`threading.Lock`	Хүлээлгийн дараалал, TA-ийн төлөв, тоолуурыг хамгаална
`students_sem`	`Semaphore(0)`	Хүлээж буй оюутны тоо. TA үүн дээр *wait* хийж унтана, оюутан *signal* өгч сэрээнэ

Объект	Төрөл	Үүрэг
`turn[i]`	`Semaphore(0)`	Оюутан бүрийн хувийн semaphore. ТА үйлчилж дуустал оюутан үүн дээр хүлээнэ
`_run`, `_stop`	`threading.Event`	Симуляцийг түр зогсоох (pause) ба бүрэн зогсоох

Thread бүр өөрийн үйлдлийг (оюутан ирсэн, суусан, буцсан, ТА сэрсэн, үйлчилж эхэлсэн, дууссан, унтсан) event болгон `queue.Queue`-д хийнэ. Tkinter-ийн үндсэн thread нь `root.after(...)`-ээр тэр дарааллыг уншиж дэлгэц дээр зурдаг. Иймээс дүрслэл нь core-ийн бодит ажиллагааг харуулж, UI thread нь core-ийн хугацаанд нөлөөлөхгүй.

1.5 Төслийн бүтэц ба хэрэгслүүд

Зүйл	Агуулга
`src/ta_core.py`	Core: *Config*, *Simulation*, thread-үүд, консол горим
`src/ui.py`	Tkinter dashboard
`src/test_ta_core.py`	Багийн 11 автомат тест (*unittest*)
`tests/`	27 edge case тест, core ба UI сценарийн скрипт, screenshot
`docs/`	Report, Manual
Хэл, сан	Python 3.14, Tk 9.0. Стандарт сан: *threading*, *queue*, *tkinter*, *unittest*
Хувилбарын удирдлага	Git, GitHub (https://github.com/B242270001/II-4-10_SleepingTeachingAssistant)

10. Дүгнэлт

10.1 Хийгдсэн ажил

Sleeping Teaching Assistant бодлогыг бодит thread, mutex, semaphore ашиглан хэрэгжүүлж, UI-аас хамааралгүй core болон Tkinter dashboard-ыг бүтээсэн. Dashboard нь оролтыг шалгаж, симуляцийг START, PAUSE, STEP, RESET-ээр удирдах, TA ба оюутнуудын хөдөлгөөнийг, тоолуур ба графикийг бодит цагт харуулах, event түүхийг шүүж, хайж, CSV болгон хадгалах боломжтой. Хэрэглэгчийн болон operational гарын авлага (Manual) бэлтгэсэн.

10.2 Туршилтын дүн

Түвшин	Хамрах хүрээ	Дүн
Багийн автомат тест	Оролтын шалгалт, зөв байдлын шинж (invariant), дээд ачаалал, pause/resume/reset	11 / 11 тэнцсэн
Edge case тест	Хязгаарын утга, ер бусын дараалал, FIFO, тоолуур, TA-ийн төлөв	27 / 27 тэнцсэн
UI сценари (S1–S11)	Хэвийн, хязгаар, алдаа, удирдлага; screenshot-той	11 / 11 тэнцсэн

Synchronization-ы гол шинжүүд бүх тохиргоонд биелсэн:

- Хүлээлгийн дараалал mutex-ээр хамгаалагдсан тул сандлын тооноос хэтэрсэн, давхцсан бичлэг гараагүй (дээд ачаалал 20 сандал, 50 оюутанд дараалал хамгийн ихдээ 20).
- TA нэг дор зөвхөн нэг оюутанд үйлчилсэн.
- FIFO дараалал баталгаажсан: эхэлж суусан оюутан эхэлж үйлчлүүлсэн.
- Semaphore-оор унтах ба сэрэх зөв ажиллаж, TA оюутангүй үед унтаж, анхны ирсэн оюутан сэрээсэн.
- Deadlock болон thread гоожилт гараагүй (олон дахин reset хийсний дараа ч thread үлдээгүй).
- Сандал 0 үед хэн ч үйлчлүүлээгүй, сандал хангалттай үед хэн ч буцаагүй.

10.3 Илэрсэн хязгаарлалт

Туршилтаар дараах хязгаарлалтууд илэрсэн. Эдгээр нь core-ийн synchronization-ы зөв байдалд нөлөөлөөгүй.

- **UI-ийн дүрслэл ачаалалтай үед хоцрох.** Олон оюутан буцдаг тохиргоонд UI нь event-үүдийг тогтмол хурдаар тоглуулдаг тул дэлгэц дээрх “үйлчлүүлсэн” тоо core-оос бага харагдана (S2: core 11, UI 1; S6: core 33, UI 1). Core-ийн тоо зөв.
- **STEP товч.** Event буфер хоосон үед core-г үргэлжлүүлдэг тул дараагийн STEP-д нэгээс олон event нэг дор орж ирэх тохиолдол ажиглагдсан.
- **reset(new_config).** Буруу тохиргоотой дуудахад эхлээд stop() хийгдэж, дараа нь тохиргоо шалгагддаг тул симуляци зогссон хэвээр үлдэнэ. UI тохиргоог урьдчилан шалгадаг тул хэрэглэгчид тохиолдохгүй.
- **Seed.** UI-д seed талбар байхгүй тул сценарийг яг ижил тоогоор давтах боломжгүй. Seed нь консол горим ба тестэд ажиллана.

- **Орчин.** Туршилтыг macOS дээр хийсэн. UI сценарийн скрипт зөвхөн macOS-д ажиллана.

10.4 Цаашид сайжруулах чиглэл

- Event буфер хуримтлагдсан үед нэг frame-д олон event боловсруулах эсвэл дүрслэх хурдыг автоматаар нэмж, UI-г core-той нийцүүлэх.
- `reset`-д эхлээд тохиргоог шалгаад дараа нь `stop()` хийх.
- UI-д seed талбар нэмж, туршилтыг яг давтах боломжтой болгох.
- Тест файлуудыг нэг хавтсанд цэгцэлж, нэг командаар ажиллуулдаг болгох.
- Өөр үйлдлийн системд (Windows, Linux) тест, UI-г шалгах.

10.5 Эцсийн дүгнэлт

Төслөөр thread, mutex, semaphore-оор нийтийн нөөцийг хамгаалах, thread-үүдийг унтуулж сэрээх зарчмыг бодитоор хэрэгжүүлж, үр дүнг нь харагдахуйц дүрсэлж, олон түвшний тестээр баталгаажууллаа. Core-ийн synchronization бүх тохиргоонд зөв ажилласан бөгөөд илэрсэн хязгаарлалтууд нь голчлон дүрслэлийн хэсэгт байна. Эдгээрийг дээрх чиглэлээр засч сайжруулах боломжтой.

Ашигласан материал

[1] Silberschatz, A., Galvin, P. B., Gagne, G. *Operating System Concepts*. 10th Edition. Wiley, 2018. Chapter 7, Programming Project 10: The Sleeping Teaching Assistant.

[2] Python Software Foundation. *threading — Thread-based parallelism*. Python 3 documentation. https://docs.python.org/3/library/threading.html

[3] Python Software Foundation. *queue — A synchronized queue class*. Python 3 documentation. https://docs.python.org/3/library/queue.html

[4] Python Software Foundation. *tkinter — Python interface to Tcl/Tk*. Python 3 documentation. https://docs.python.org/3/library/tkinter.html

[5] Python Software Foundation. *unittest — Unit testing framework*. Python 3 documentation. https://docs.python.org/3/library/unittest.html

[6] Багийн GitHub repository: https://github.com/B242270001/II-4-10_SleepingTeachingAssistant

[7] II-4-10 баг. *Туршилт, Үр дүн* (testing_results.pdf). Туршилтын тайлан, 2026.10.28.

[8] II-4-10 баг. *Хэрэглэгчийн болон Operational гарын авлага* (II-4-10_Manual.pdf).